Student Lecture Notes

📌 PART 1: SHORT REVISION SHEET (Condensed Study Notes)

1. Web Communication and Protocols

What is a Protocol?

A **protocol** is a set of rules that define how communication occurs between systems.

Examples of internet protocols:

- **IP** – Internet Protocol
- **TCP** – Transmission Control Protocol
- **UDP** – User Datagram Protocol
- **HTTP** – HyperText Transfer Protocol

Protocols ensure that devices understand:

- How to send data
- How to receive data
- How to interpret data

2. HTTP and HTTPS

2.1 HTTP (HyperText Transfer Protocol)

- Application-level protocol
- Used for transferring web data
- Stateless (each request is independent)
- Data is sent in **clear text**

⚠ Risk: Anyone intercepting traffic can read the data.

2.2 HTTPS (HTTP Secure)

HTTPS = HTTP + Encryption (via SSL/TLS certificates)

- Data is encrypted
- Certificates are issued by Certificate Authorities (CA)
- Certificates have expiration dates

✓ Protects sensitive information (passwords, credit cards)

✓ Required for secure modern applications

3. Client–Server Architecture

The web follows a **Client–Server model**.

Client

- Initiates communication
- Sends requests
- Examples: Browser, mobile app, Postman

Server

- Receives request
- Processes request
- Sends response

Communication pattern:

Client → Request → Server

Server → Response → Client

4. URL – Uniform Resource Locator

A URL identifies a resource on the web.

Example structure:

https://example.com/products/shoes?color=black&size=42#details

4.1 URL Components

1. Schema (Protocol)

- http
- https
- ftp

2. Domain

- example.com

3. Path

- /products/shoes
  Indicates resource location.

4. Query String

- ?color=black&size=42
  Used for filtering.
- Key-value pairs
- Separated by "&"

5. Fragment

- #details
  Points to a specific HTML element.
- Not processed by servers in APIs.

5. HTTP Request

A request is like an envelope sent from client to server.

5.1 Request Structure

1. **Header**
   - Metadata
   - Contains key-value pairs
   - Example: Content-Type, Accept
2. **Body**
   - Actual data being sent
   - May be empty (e.g., GET request)
3. **HTTP Method (Verb)**

5.2 HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create new resource
PUT	Update existing resource

DELETE	Remove resource
HEAD	Same as GET but without body
OPTIONS	Shows allowed methods

6. HTTP Response

A response is the server's reply.

6.1 Response Structure

1. **Header**
2. **Body**
3. **Status Code**

Difference:

- Request contains HTTP Method.
- Response contains Status Code.

7. MIME Types

MIME = Multipurpose Internet Mail Extensions

Format:

type/subtype

Examples:

- text/html
- application/json
- image/png

Important Headers

Accept

Client tells server:

"What format I want."

Content-Type

Specifies:

- Format of request body (in request)
- Format of response body (in response)

If server cannot interpret body →

415 Unsupported Media Type

This process is called **Content Negotiation**.

8. CRUD and HTTP Mapping

CRUD = Create, Read, Update, Delete

Operation	**HTTP Method**
Create	POST
Read	GET
Update	PUT

Delete	DELETE

Important for API design.

9. Safe vs Idempotent Methods

Idempotent

Multiple identical requests → same result.

Examples:

- GET
- PUT
- DELETE

Safe

Does NOT modify data.

Safe methods:

- GET
- HEAD

Unsafe methods:

- POST
- PUT
- DELETE

10. HTTP Status Codes

Status codes are 3-digit numbers grouped into 5 categories.

1xx – Informational

Temporary responses.

Rarely used in APIs.

2xx – Success

- 200 OK
- 201 Created
- 202 Accepted
- 204 No Content

3xx – Redirection

- 301 Moved Permanently

4xx – Client Errors

Indicates client mistake.

Common examples:

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 405 Method Not Allowed

- 406 Not Acceptable
- 409 Conflict
- 410 Gone
- 415 Unsupported Media Type
- 422 Unprocessable Entity
- 429 Too Many Requests

5xx – Server Errors
Indicates server failure.
Examples:

- 500 Internal Server Error
- 501 Not Implemented
- 503 Service Unavailable

11. Cookies
HTTP is **stateless**.
Cookies allow servers to maintain state.
Cookie Properties

- Name
- Value
- Domain
- Expiration Date
- HTTPOnly
- Secure
- SameSite
- Session flag

Same-Site vs Cross-Site

- Same-site request → safer
- Cross-site request → higher risk

Cookies are used for:

- Authentication
- Session tracking
- "Remember me" functionality

12. Key Concepts to Remember
Students must clearly understand:

1. Difference between HTTP and HTTPS
2. Client–Server communication model
3. URL structure
4. Request vs Response structure
5. HTTP methods and their purposes
6. MIME types and content negotiation
7. CRUD mapping
8. Safe vs Idempotent methods

9. Status code categories
10. Role of cookies

📌 **PART 2: PRACTICE QUESTIONS (With Answers)**

Multiple Choice Questions

Q1: Which HTTP method is used to retrieve data?

A) POST
B) PUT
C) GET
D) DELETE

✅ Answer: C

Q2: Which status code means "Resource created"?

A) 200
B) 201
C) 204
D) 400

✅ Answer: B

Q3: Which method is NOT idempotent?

A) GET
B) PUT
C) DELETE
D) POST

✅ Answer: D

Q4: Which header tells the server what format the client expects?

A) Content-Type
B) Accept
C) Authorization
D) Host

✅ Answer: B

Q5: 404 indicates:

A) Server error
B) Resource not found
C) Redirect
D) Unauthorized

✅ Answer: B

Short Answer Questions

Q1: What is the difference between HTTP and HTTPS?
Answer: HTTPS encrypts data using SSL/TLS certificates, while HTTP sends data in plain text.

Q2: Define idempotent method.
Answer: An HTTP method is idempotent if multiple identical requests produce the same result.

Q3: What is Content Negotiation?
Answer: The process where client and server agree on the format of the data using Accept and Content-Type headers.

📌 PART 3: COMPARISON SUMMARY TABLES

HTTP vs HTTPS

Feature	HTTP	HTTPS
Encryption	No	Yes
Security	Low	High
Uses Certificates	No	Yes
Default Port	80	443

Request vs Response

Feature	Request	Response
Sent by	Client	Server
Contains Method	Yes	No
Contains Status Code	No	Yes
Contains Body	Optional	Optional

Safe vs Idempotent

Property	Safe	Idempotent
Modifies Resource	No	May or may not
Example	GET	GET, PUT
POST	No	No

4xx vs 5xx Errors

Category	Who Made Error?
4xx	Client
5xx	Server

📌 PART 4: EXAM-STYLE LONG ANSWER QUESTIONS

Question 1:
Explain the structure of an HTTP request and response with examples.

Question 2:
Discuss HTTP status code categories and provide examples for each.

Question 3:
Explain CRUD operations and how they map to HTTP methods.

Question 4:
Compare HTTP and HTTPS in terms of security and functionality.

Question 5:
Explain Safe and Idempotent methods with examples.

Question 6:
Discuss the role of cookies in web communication and explain their key attributes.